数据库概论作业（第八章-第十二章）

毛川 2300013218

第八章 事务

1. 当一个事务中的某条SQL语句遇到运行错误时，该事务该如何进行善后处理？

> 先撤销该语句已经造成的影响；然后根据错误严重程度选择 ROLLBACK 整个事务，或回滚到事先设置的 SAVEPOINT；最后释放相关锁和资源。

2. 阐述事务一致性和数据完整性的对比。

> 数据完整性是数据库本身需要满足的约束，例如主键、外键、唯一性、非空、检查约束等。事务一致性是指一个事务执行前数据库是一致状态，事务正确执行后数据库仍然是一致状态。

3. 阐述一致性和原子性的对比。

> 原子性强调事务内部操作要么全做，要么全不做。一致性强调事务执行完成后，数据库必须从一个合法状态转移到另一个合法状态。

4. n个事务， T_i 有k_i 条指令，则可能的并发调度有多少个？

> $$\frac{(\sum k_i)!}{\prod k_i!} >$$
> ∏ki!(∑ki)! >

5. 列举一些必须要重复读的业务场景。

> 银行转账中多次读取账户余额；
> 库存扣减时多次读取商品库存；
> 考试报名系统中多次读取剩余名额；
> 订单支付过程中多次读取订单状态和金额；

6. 无级联调度会发生哪些不一致现象？

丢失修改；
不可重复读；
读幻觉；
读错误数据；

7. 要防止发生丢失修改，则事务至少要设置在哪个隔离性级别上？

Repeatable Read；因为 Read Committed 只保证事务不能读到未提交数据，也就是防止脏读；但它不保证同一个数据在事务执行期间不被别人修改。因此，如果程序采用“先读出来，在应用层计算，再把计算结果写回去”的方式，就可能发生丢失修改。

8. 要防止发生幻象，则事务至少要设置在哪个隔离性级别上？

至少需要 Serializable

9. 为什么在快照隔离中，读写操作不会产生冲突？

因为读操作读取的是事务开始时或语句开始时的历史版本快照，而不是直接读取正在被写的最新版本。

10. 什么是快照隔离中的写偏斜现象？要避免发生写偏斜，事务至少要设置在哪个隔离性级别上？

写偏斜是指两个事务都基于同一个旧快照读取数据，判断约束仍然成立，然后分别修改不同的数据项，最终合并后破坏约束。

11. 语句级快照隔离RCSI更接近RC，而事务级快照隔离SI更接近RR，对吗？它们彼此的差别是什么？

RCSI 是语句级快照隔离。每条语句看到的是该语句开始时的已提交版本，因此同一事务中两条 SELECT 可能看到不同快照。所以它更接近 Read Committed。
SI 是事务级快照隔离。整个事务期间看到的是事务开始时的同一个快照，因此同一事务内重复读取通常一致。所以它更接近 Repeatable Read。

12. 什么是MySQL的当前读和快照读？

快照读读取 MVCC 生成的历史版本，不加锁。
当前读读取记录的最新版本，并且通常会加锁。

13. 构造满足如下复合条件的调度：

记号说明：
`r1(A)` 表示事务 T1 读 A；
`w1(A)` 表示事务 T1 写 A；
`c1` 表示 T1 提交；
`a1` 表示 T1 回滚。

(a) **无级联 + 非冲突可串行化**

$$w_1(A),\ w_2(A),\ w_2(B),\ w_1(B),\ c_1,\ c_2$$

w1(A), w2(A), w2(B), w1(B), c1, c2
没有读未提交数据，所以无级联。
冲突图中有：

$$T_1 \rightarrow T_2$$

T1 → T2
来自 `w1(A)` 在 `w2(A)` 前；又有：

$$T_2 \rightarrow T_1$$

T2 → T1
来自 `w2(B)` 在 `w1(B)` 前。
存在环，所以非冲突可串行化。

(b) **级联 + 冲突可串行化**

$$w_1(A),\ r_2(A),\ w_2(B),\ c_1,\ c_2$$

w1(A), r2(A), w2(B), c1, c2
`r2(A)` 读取了 T1 尚未提交的写入，因此是级联风险调度。
冲突方向只有：

$$T_1 \rightarrow T_2$$

T1 → T2
所以冲突可串行化，等价于串行顺序 (T_1,T_2)。

(c) **无级联 + 非视图可串行化**

$$r_1(A),\ w_2(A),\ c_2,\ w_1(A),\ c_1$$

r1(A), w2(A), c2, w1(A), c1
`r1(A)` 读的是初始值，没有读未提交数据，所以无级联。

但它非视图可串行化。原因是：
T1 读到了 A 的初始值，同时最终写 A 的也是 T1。若串行顺序是 (T_1,T_2)，最终写者应为 T2；若串行顺序是 (T_2,T_1)，T1 又不可能读到 A 的初始值。因此不存在等价串行调度。

(d) **级联 + 视图可串行化**

$$w_1(A),\ r_2(A),\ w_2(B),\ c_1,\ c_2$$

w1(A), r2(A), w2(B), c1, c2
`r2(A)` 读取了 T1 尚未提交的 A，因此是级联风险调度。

它视图可串行化，因为它等价于串行顺序：

$$T_1 \rightarrow T_2$$

T1 → T2
T2 读到的是 T1 写的 A，最终 A 的写者是 T1，B 的写者是 T2，与串行调度 (T_1,T_2) 一致。

第九章：并发控制问答题

1. 2PL可以保证调度是可串行化的吗?2PL可以保证调度是无级联的吗?2PL可以保证调度避免不可重复读吗?哪个事务隔离性级别肯定满足2PL?

2PL 可以保证调度是冲突可串行化的：
两段锁协议要求事务分为加锁阶段和解锁阶段：一旦释放锁，就不能再申请新锁。因此所有满足 2PL 的调度都是冲突可串行化调度。

基本 2PL 不一定保证无级联：
因为基本 2PL 允许事务在提交前释放写锁。其他事务可能读到它尚未提交的数据，从而产生级联回滚。要保证无级联，通常需要 严格 2PL：写锁一直持有到事务结束。

2PL 可以避免不可重复读：
因为满足 2PL 的调度是可串行化的，而可串行化调度不会出现不可重复读。

2. 升级锁和重新申请锁有区别吗? 在哪个隔离性级别下会出现锁转换? 避免转换死锁的关键是什么? 持有U锁可以对数据执行什么操作? U和S相容吗?U和U相容吗?

升级锁是指事务已经持有较弱的锁，然后在不释放原锁的情况下申请更强的锁，例如： S→X。
重新申请锁是先释放原来的锁，再重新申请新锁。二者不同。重新申请中间有空隙，其他事务可能修改数据，也可能破坏 2PL。

锁转换常出现在 Repeatable Read 和 Serializable 这类长期持有读锁的隔离级别下。因为事务先读数据时持有 S 锁，之后又要修改该数据，就需要从 S 锁升级到 X 锁。

避免锁转换的关键：区分只读和先读后写

持有 U 锁时，可以读取数据，并表示“我以后可能要更新它”。但 U 锁本身通常不能直接执行写操作，真正写之前仍要升级为 X 锁。U 与 S 相容，但是 U 与 U 不相容。

3. 如何权衡封锁粒度大小的利弊? 为什么需要支持多粒度锁? 如何避免不同粒度的锁之间的冲突?

封锁粒度越小，例如行锁：并发度高；冲突少；但锁数量多，锁管理开销大；也更容易因为大量锁请求增加死锁机会。

封锁粒度越大，例如表锁：锁数量少；管理开销小；死锁检测也更简单；但并发度低，容易阻塞无关操作。

两种锁各有优劣，同一个系统中，有些事务只访问一行，适合行锁；有些事务扫描整张表，适合表锁。

为避免不同粒度锁之间冲突，需要使用 意向锁。例如事务想对子节点加 X 锁，必须先在父节点上加 IX 锁。这样其他事务想对整个表加 S 锁或 X 锁时，可以通过父节点上的意向锁判断下面是否已有冲突锁。

4. 和IS相容的锁有哪些? 和IX相容的锁有哪些? SIX适合什么应用场合? 它和哪些锁相容?

与 IS 相容的锁有：IS, IX, S, SIX
与 IX 相容的锁有：IS, IX
SIX 表示：对当前对象整体加 S 锁，同时对子对象有加 X 锁的意向。也就是“我要读整个表，并可能修改其中一些行”。
SIX 适合这种场景：事务需要扫描整张表，同时更新其中部分记录。

5. 死锁发生的条件有哪些? 从预防死锁的角度，列举一些预防死锁的措施，说明它们分别破坏了死锁发生的哪个条件?

互斥条件；占有并等待；不可抢占；循环等待。

常见预防措施：

1. 一次性申请所有锁，破坏“占有并等待”。事务开始时一次拿到所有需要的锁，拿不到就不执行。
2. 按固定顺序加锁，破坏“循环等待”。例如所有事务都必须按 A、B、C 的顺序申请锁，就不容易形成环。
3. 抢占锁或回滚事务，破坏“不可抢占”。当系统发现某事务阻塞其他事务时，可以强制回滚其中一个事务。
4. 使用超时机制，也近似破坏“不可抢占”或“无限等待”。等待超过一定时间就主动回滚。

6. 讨论事务隔离性级别，事务粒度、封锁粒度和死锁发生几率的关系。

1. 隔离级别越高，持有锁的时间长，范围大，容易死锁；
2. 事务粒度大，死锁概率更高
3. 封锁粒度越细，锁的数量通常越多，往往更加容易死锁。

7. 讨论影响事务吞吐率的一些措施: 隔离性级别，事务粒度、封锁粒度、绑定连接，通过消息队列来解耦业务。

隔离级别：隔离级别越低，锁冲突越少，吞吐率通常越高，但一致性风险更大。Serializable 最安全，但吞吐率最低。
事务粒度：事务越短，吞吐率通常越高。
封锁粒度：细粒度锁提高并发，但管理开销大；粗粒度锁管理简单，但降低并发。需要根据业务访问模式选择。
绑定连接：如果多个操作必须共享同一个事务上下文，可以绑定到同一连接或同一事务上下文，减少连接切换和事务管理成本。但连接长期占用也会降低连接池效率。
消息队列解耦业务：把长事务拆成短事务，把非核心操作异步化。可以减少锁持有时间，提高吞吐率，但代价是系统变成最终一致性。

8. 列举可以提高事务吞吐率的措施。

降低不必要的隔离级别；
缩短事务时间；
减少事务中访问的数据量；
避免长事务；
使用行锁而不是表锁；
按固定顺序访问数据，减少死锁；

9. 什么是活锁? 如何避免活锁?

活锁是指事务没有形成死锁，但某个事务一直得不到执行机会，反复被其他事务插队或回滚，长期处于等待状态。

可以采用更加公平的调度算法，使长期等待的事务更有可能被接下来执行。

10. 在时间戳协议中，如何确定一个读写操作是正常的? 什么是跳过的写? 提交位的作用是什么?

1. 如果 $TS(T_i) < WT(Q)$TS(Ti) < WT(Q)，说明 T_iTi 太旧，它想读的数据已经被更新的事务覆盖，读操作不正常，应拒绝或回滚。否则读操作正常
2. 如果 $TS(T_i) < WT(Q)$TS(Ti) < WT(Q) 说明 Q 已经被更新的事务写过，当前写入已经过时。按照 Thomas 写规则，可以不回滚，而是直接忽略这次写。这就是跳过的写。
3. 提交位用于标记某个版本是否已经提交。它的作用是防止事务读到未提交版本，从而避免脏读和级联回滚。读事务如果遇到提交位为假的版本，通常需要等待该版本提交或回滚后再决定是否读取。

11. 在有效性检查协议中，在确认一个事务时，它的读写集合需要与之前已经得到确认的事务的读写集合进行相交性判定。 什么时候它的读写集合都要与之前已确认事务的写集合进行判定? 什么时候只需要它的读集合与之前已确认事务的写集合进行判定?

有效性检查协议, 设当前要确认的事务是 (T_i)，之前已经确认的事务是 (T_j)。

如果 (T_j) 已经通过确认，但它的写阶段还没有完成，也就是 (T_j) 的写阶段可能与 (T_i) 的确认或写阶段重叠，则需要检查两类冲突：$RS(T_i) \cap WS(T_j) = \varnothing$RS(Ti) ∩ WS(Tj) = ∅, $WS(T_i) \cap WS(T_j) = \varnothing$WS(Ti) ∩ WS(Tj) = ∅, 这时当前事务的读集合和写集合都要与之前已确认事务的写集合判定。

如果 (T_j) 和 (T_i) 有重叠，但 (T_j) 已经完成写阶段，那么只需要检查：$RS(T_i) \cap WS(T_j) = \varnothing$RS(Ti) ∩ WS(Tj) = ∅, 也就是当前事务的读集合不能和之前事务的写集合相交。原因是：如果相交，说明 (T_i) 可能读到了被 (T_j) 修改前的旧值。

12. 在MVCC中，读视图结构是怎样的? 可见性算法思想是怎样的? RC和RR的读视图的区别是什么?

在 MySQL InnoDB 中，Read View 通常包含几个核心信息：

当前活跃事务 ID 列表；m_ids
当前活跃事务中的最小 ID；min_trx_id
系统将来要分配的下一个事务 ID；max_trx_id
创建该 Read View 的事务 ID；creator_trx_id
每条记录版本上有一个事务 ID，记为：trx_id

可见性判断大致如下：

如果版本的 trx_id = creator_trx_id 说明是当前事务自己改的，可见。
如果 trx_id < min_trx_id 说明该版本在 Read View 创建前已经提交，可见。
如果 trx_id > max_trx_id 说明该版本是在 Read View 创建后才出现的，不可见。
如果 min_trx_id < trx_id < max_trx_id 则看它是否在活跃事务列表 (m_ids) 中。
如果在 (m_ids) 中，说明创建快照时该事务还没提交，不可见。
如果不在 (m_ids) 中，说明创建快照时它已经提交，可见。

如果当前版本不可见，就沿着 undo log 版本链向旧版本查找，直到找到一个对当前 Read View 可见的版本。

RC 和 RR 的 Read View 区别：

在 **Read Committed** 下，每条普通 `SELECT` 语句都会生成一个新的 Read View。因此同一事务内两次读取，可能看到不同的已提交结果。

在 **Repeatable Read** 下，事务第一次执行快照读时生成 Read View，之后事务内的快照读复用这个 Read View。因此同一事务内多次读取通常结果一致。

第十章：恢复控制

1. 恢复的目标有哪些?恢复的措施有哪些?故障类型有哪些? 备份的类型有哪些?

恢复目标：保证事务的原子性和持久性。也就是：已提交事务的结果不能丢失；未提交事务的影响必须撤销；数据库恢复到一致状态。

恢复措施：日志、检查点、备份、归档日志、影子分页、复制、RAID、故障检测与重启恢复算法。

故障类型主要有：
事务故障：事务自身出错、死锁回滚、约束违反等。
系统故障：断电、操作系统崩溃、DBMS 崩溃，内存丢失但磁盘仍在。
介质故障：磁盘损坏、数据文件丢失。
人为或应用故障：误删表、错误更新、程序逻辑错误。
灾难故障：机房损毁、区域性故障。

备份类型主要有：完全备份；增量备份；差异备份；物理备份；逻辑备份；冷备份；热备份；本地备份；远程备份；快照备份；归档日志备份。

2. 什么是影子分页技术?

影子分页区别于日志，类似 copy-on-write。系统维护两张页表：一张是影子页表，表示事务开始前的数据库状态；一张是当前页表，表示事务执行中的新状态。
事务修改某个页时，不直接覆盖原页，而是把修改写到新页，并让当前页表指向新页。影子页表保持不变。提交时，只需要原子地把数据库根指针从影子页表切换到当前页表。回滚时，直接丢弃当前页表和新页，继续使用影子页表。

优点是恢复简单；缺点是页复制开销大、容易产生碎片、并发控制复杂。

3. 日志基本的恢复操作是什么? 什么是WAL原则?

日志恢复的基本操作是：
UNDO：撤销未提交事务的影响。
REDO：重做已提交事务的影响。

WAL，即 Write-Ahead Logging，日志先写原则。核心要求是：
数据页写入磁盘之前，对应的日志必须先写入稳定存储。事务提交成功之前，该事务的提交日志和必要更新日志必须先落盘。
简单说：先写日志，再写数据。

4. 日志类型有哪些?它们的比较维度有哪些?

常见日志类型有：
UNDO 日志：记录旧值，用于撤销。
REDO 日志：记录新值，用于重做。
UNDO/REDO 日志：同时记录旧值和新值，既能撤销又能重做。
物理日志：记录具体页、偏移和值。
逻辑日志：记录逻辑操作，如“给账户 A 加 100”。

比较维度主要有：
是否支持 UNDO；
是否支持 REDO；
记录旧值还是新值；
日志量大小；
恢复速度；
运行时开销；
是否要求提交时强制刷数据页；
是否允许未提交脏页落盘；
是否适合高并发事务系统。

5. 各种故障类型的恢复过程是怎样的?

事务故障：只回滚该事务。根据日志从后向前执行 UNDO，撤销该事务已经执行的操作。
系统故障：内存丢失，磁盘仍在。恢复时先找检查点，然后 REDO 已提交但可能未落盘的事务，再 UNDO 未提交事务。
介质故障：磁盘数据损坏。先用最近一次备份恢复数据库文件，再用归档日志和在线日志 REDO 到故障前状态，最后撤销未提交事务。
人为误操作：通常需要基于备份、日志、闪回、时间点恢复等手段恢复到误操作前的状态。

6.什么是检查点?它的作用是什么?

检查点是数据库周期性在日志中记录系统当前状态的一种机制。
检查点通常记录：

1. 当前活跃事务；
2. 脏页集合；
3. 各事务最近日志位置；
4. 各脏页最早使其变脏的日志位置。

作用是减少恢复时需要扫描的日志范围。没有检查点时，系统崩溃后可能要从很早的日志开始恢复。有检查点后，恢复可以从最近检查点附近开始，大大缩短恢复时间。
检查点分为：

1. 静态检查点：暂停事务，刷盘后记录检查点，简单但影响运行。
2. 模糊检查点：不暂停事务，只记录恢复所需状态，ARIES 使用这种思想。

7.针对事务的脏写是否可以落盘和提交写是否必须落盘所形成的各种缓冲区管理策略，对日志类型的需求是怎样的? 如何评估不同策略对于系统运行时的性能影响?

是否允许**未提交事务的脏页落盘**：STEAL / NO-STEAL。
事务提交时是否必须把其修改的数据页刷盘：FORCE / NO-FORCE。

四种组合如下：

策略	未提交脏页可落盘?	提交时数据页必须落盘?	需要的日志能力
NO-STEAL + FORCE	否	是	理论上不需要 UNDO/REDO
NO-STEAL + NO-FORCE	否	否	需要 REDO
STEAL + FORCE	是	是	需要 UNDO
STEAL + NO-FORCE	是	否	需要 UNDO + REDO

性能角度：

FORCE 会增加提交延迟，因为提交时必须刷数据页，随机 I/O 多，吞吐率低。
NO-FORCE 提交快，只需要刷日志，运行性能较好，但恢复时需要 REDO。
NO-STEAL 要求未提交脏页不能被换出，对缓冲区压力大。
STEAL 缓冲区利用率高，但恢复逻辑复杂，需要 UNDO。

8.Aries算法相应的页结构、ATT、DPT是怎样的?什么是补偿日志?

ARIES 中，每个数据页通常有：pageLSN：表示该页已经包含的最新日志记录的 LSN。恢复时用它判断某条日志是否已经反映到该页中。
日志记录通常有：LSN；事务 ID；类型；pageID；prevLSN；redo 信息；undo 信息。

ATT 是 Active Transaction Table，活跃事务表。主要记录：事务 ID；事务状态；lastLSN，即该事务最近一条日志的 LSN。

DPT 是 Dirty Page Table，脏页表。主要记录：脏页 pageID；recLSN，即使该页第一次变脏的日志 LSN。

补偿日志，即 CLR，Compensation Log Record，是在 UNDO 时写入的日志。
它表示“某个操作已经被撤销”。CLR 有几个特点：CLR 只需要 REDO，不再被 UNDO；CLR 中包含 undoNextLSN，用于指出下一步应该继续撤销哪条日志；如果 UNDO 过程中系统再次崩溃，恢复时可以根据 CLR 继续执行，避免重复撤销错误。

9. Aries算法的三个阶段是怎样的?各个阶段的处理起点是怎样的?

第一阶段：Analysis，分析阶段
起点：从最近一次检查点开始扫描日志。
目的：
重建 ATT；
重建 DPT；
判断哪些事务是 winner，哪些事务是 loser。

第二阶段：Redo，重做阶段
起点：从 DPT 中最小的 recLSN 开始。
ARIES 的 REDO 思想是：repeat history
也就是尽量重复崩溃前数据库已经做过的历史操作，包括已经提交事务的操作，也包括后来可能要被撤销的未提交事务操作。
是否真的重做某条日志，要看 pageLSN：如果页面的 pageLSN 已经大于等于该日志的 LSN，说明该操作已经反映在页中，不必重做。
否则执行 REDO。

第三阶段：Undo，撤销阶段
起点：从 ATT 中所有 loser 事务的 lastLSN 开始，按 LSN 从大到小回退。
处理方式：
沿着每个事务的 prevLSN 链向前撤销；
每撤销一步，写一条 CLR；
直到事务所有需要撤销的操作都撤销完；
最后写入事务结束日志。

10.扩展讨论:agent harness工程和数据库恢复控制的技术契合点。

agent harness 可以理解为管理智能体执行过程的工程框架，包括任务调度、工具调用、状态保存、失败重试、日志记录、回滚补偿、结果提交等。

它和数据库恢复控制有不少技术契合点。

第一，二者都需要**日志化执行过程**。
数据库用 WAL 记录事务更新；agent harness 也应记录每一步工具调用、输入、输出、状态变化和副作用，方便失败后恢复或审计。

第二，二者都需要区分**已提交状态和中间状态**。
数据库事务提交前的修改不能被当作最终结果；agent 执行中间生成的草稿、临时文件、半完成 API 调用，也不应直接暴露为最终状态。

第三，二者都需要**幂等与重试机制**。
数据库 REDO 要尽量做到可重复执行；agent harness 中的工具调用也应设计成可幂等重试。例如同一个任务重试时，不应重复扣款、重复发邮件、重复创建订单。

第四，二者都需要**补偿机制**。
数据库用 UNDO 或 CLR 撤销未完成事务；agent 工程中如果一个多步骤任务执行到一半失败，也需要补偿动作。例如取消已创建资源、删除临时文件、撤销外部系统变更。

第十一章：数据库存储

1.磁盘顺序读和随机读的性能差异对数据库设计有哪些方面的影响?

顺序读远快于随机读，所以数据库设计会尽量把访问变成连续 I/O。主要影响包括：索引采用 B+ 树，便于范围扫描；表、索引、页、区尽量连续分配；

2.各个RAID级别设计机制(冗余和拆分)的差异是怎样的?数据库该如何选择合适的RAID级别?

RAID 的核心机制有两个：拆分 striping 和 冗余 redundancy。
RAID 0：只拆分，不冗余。性能高，但一块盘坏就丢数据。
RAID 1：镜像冗余。可靠性高，读性能较好，写入要写两份。
RAID 5：条带化 + 分布式校验。空间利用率高，可容忍一块盘坏，但小随机写有校验写放大。
RAID 6：类似 RAID 5，但有双校验，可容忍两块盘坏，写开销更大。
RAID 10：先镜像再条带化。可靠性和随机 I/O 性能较好，但空间利用率约 50%。

3.MySQL 和 SQL Server 的缓冲区管理策略是如何具体实现的?MySQL 的changebuffer 和 double write buffer 的作用是什么?

MySQL InnoDB 使用 buffer pool 缓存表和索引页。buffer pool 是按页管理的，并使用 LRU 变体；MySQL 文档说明其采用链表和 midpoint insertion，把新读入页放到 LRU 列表中间，以减少全表扫描污染缓存。

SQL Server 使用 buffer pool / buffer cache 缓存数据页和索引页。SQL Server 的页大小是 8 KB，buffer manager 负责把磁盘页读入缓存，并把修改过的页写回磁盘；页会保留在缓存中直到需要腾出空间。

change buffer 是 InnoDB 针对二级索引修改的缓冲结构。当二级索引页不在 buffer pool 中时，先把 INSERT、UPDATE、DELETE 对二级索引的修改缓存起来，等以后该页被读入时再合并，从而减少随机 I/O。
doublewrite buffer 用于防止页写入过程中发生 torn page。InnoDB 在把脏页写到真正的数据文件位置之前，先写入 doublewrite buffer；如果崩溃发生在页写一半时，恢复时可以从 doublewrite buffer 找到完整页副本。

4.数据页结构和数据行结构是怎样?

数据页是数据库 I/O 的基本单位。典型页结构包括：

1. 页头：页号、页类型、校验信息、LSN、空闲空间指针等；
2. 记录区：存放实际行记录；
3. 空闲空间：用于插入和更新；
4. 槽数组或页目录：保存行在页内的偏移，便于定位行；
5. 页尾：校验、结束标记等。
   SQL Server 中，页是数据文件 I/O 的基本单位，普通页大小为 8 KiB；页尾部有 slot array，每个槽保存对应行在页内的偏移。

数据行通常包括：

1. 行头：状态位、长度、事务信息等；
2. 固定长度字段；
3. 变长字段目录；
4. 变长字段数据；
5. NULL 位图；
6. 可能的溢出页指针。
   在 InnoDB 中，表数据和二级索引都组织在 B-tree 结构中；聚簇索引记录包含完整行，二级索引记录包含索引列和主键列；过长的变长列可能放到 overflow page。

5.B树和B+树的区别是什么?

B 树：内部节点和叶子节点都可以存放数据记录。查找命中内部节点时可以提前结束。
B+ 树：只有叶子节点存放完整数据或数据指针；内部节点只存键值和子指针。叶子节点通常按键值顺序链起来。

6. 位图索引和位片索引是如何构造的?它们是如何支持查询的?

位图索引 bitmap index：对某个低基数字段的每个取值建立一个位图。位图长度等于表中行数。某行取该值，则对应位置为 1，否则为 0。每个取值对应一个 0/1 向量，查询条件通过 AND、OR、NOT 快速组合。

位图索引 bitmap index：对某个低基数字段的每个取值建立一个位图。位图长度等于表中行数。某行取该值，则对应位置为 1，否则为 0。

7.按列存储的优点是什么?如何理解 Duckdb的向量化执行以及零拷贝技术?

列存储优点：

1. 只读取查询需要的列，减少 I/O；
2. 同一列数据类型一致，压缩率高；
3. 适合聚合、扫描、过滤、分组；
4. 更容易做 SIMD、向量化执行；
5. 适合 OLAP，不太适合频繁单行更新的 OLTP。

DuckDB 向量化执行：DuckDB 而是一批一批处理，它用 Vector 表示执行过程中的列向量，用 DataChunk 表示一组向量；默认向量大小通常是 2048 个 tuple。这样可以减少函数调用开销，提高 CPU cache 和 SIMD 利用率。
零拷贝 zero-copy：零拷贝是指 DuckDB 与外部数据格式，如 Apache Arrow，交互时尽量不复制数据，而是直接在已有内存布局上执行查询或交换数据。这样减少内存复制成本，适合 Python/R 数据分析场景。DuckDB 官方文章明确提到其与 Arrow 的 zero-copy integration。

8.列举几种数据库里面的多维索引类型?

常见多维索引包括：
R 树；
R+ 树；
R* 树；
KD 树；
Quad Tree，四叉树；
SP-GiST；
Z-order / Morton 编码索引；

9.LSM树的思想是什么?相比B+树的优点是什么?为什么它需要配合Bloomfilter?

LSM 树的核心思想是：把随机写转化为顺序写。写入时先写 WAL，再写内存表 MemTable；MemTable 满后刷成磁盘上的有序文件 SSTable；后台通过 compaction 把多个 SSTable 合并；读取时可能要查 MemTable、immutable MemTable 和多个 SSTable。

相比 B+ 树，LSM 树的优点是：

1. 写入吞吐高；
2. 随机写少，顺序写多；
3. 适合 SSD 和高写入场景；
4. 批量写、日志型写入性能好；
5. 适合键值存储、日志流、时序数据等。

缺点是：读放大；空间放大；compaction 会带来写放大和后台抖动。

LSM 树需要配合 Bloom filter，因为一个 key 可能存在于多个 SSTable 中。没有 Bloom filter 时，点查询可能要查很多文件。Bloom filter 可以快速判断“某个 SSTable 一定不包含该 key”或“可能包含该 key”，从而减少无效磁盘查找。

第十二章：数据库查询处理

1.数据库查询处理所包含的完整步骤有哪些?

1. 查询解析：检查 SQL 语法是否正确。
2. 语义检查：检查表名、列名、权限、数据类型是否正确。
3. 查询重写：把 SQL 转换成更适合优化的形式，例如视图展开、子查询改写、谓词下推、常量折叠。
4. 生成逻辑查询计划：把 SQL 转换成关系代数表达式，例如选择、投影、连接、分组、排序。
5. 查询优化：选择更优的执行方案，包括访问路径选择、连接顺序选择、连接算法选择等。
6. 生成物理执行计划：确定具体使用索引扫描、全表扫描、嵌套循环连接、哈希连接、排序聚合等物理算子。
7. 执行计划：执行器按照物理计划访问数据、计算结果并返回给用户。

2.理解两种基于规则的和基于代价的查询优化策略。

基于规则的优化，RBO：根据固定规则改写查询，不依赖精确代价估计。常见规则有：

1. 选择下推；
2. 投影下推；
3. 先做过滤再连接；
4. 把笛卡尔积转成连接；

优点是简单、稳定。缺点是不能很好适应数据分布变化。

基于代价的优化，CBO：优化器为不同执行计划估算代价，然后选择总代价最低的计划。估算依据包括：

1. 表大小；
2. 索引情况；
3. 选择率；
4. 基数估计；

优点是更智能，能根据数据规模和统计信息选择计划。缺点是依赖统计信息，如果统计信息不准确，可能选错计划。

3.看懂各种访问方式的具体执行计划。

全表扫描 Table Scan：逐页读取整张表。适合小表，或查询需要返回大量数据。
索引扫描 Index Scan：按索引顺序扫描较大范围的数据。适合范围查询、排序、分组。
索引查找 Index Seek：利用索引快速定位少量记录。适合等值查询、高选择率条件。
聚簇索引扫描 Clustered Index Scan：扫描聚簇索引叶子节点，本质上接近扫描整张表。

4.理解各种算子的物理实现策略，主要是各种连接实现策略，它们的适用场景。

常见物理算子包括：
选择算子：过滤满足条件的行。
投影算子：选择需要的列。
排序算子：对结果排序。
聚合算子：执行 COUNT、SUM、AVG、GROUP BY。
连接算子：把多张表按条件组合起来。

1. 嵌套循环连接 Nested Loop Join
   思想是：
   外表取一行，内表查找匹配行；重复执行。
   适用场景：
   外表很小；
   内表连接列上有索引；
   返回结果较少；
   OLTP 点查询较多。
   代价大致是：
   外表行数×内表查找代价

   如果内表没有索引，Nested Loop 可能非常慢。

2. 块嵌套循环连接 Block Nested Loop Join
   不是一行一行扫描外表，而是一次读入外表的一批页，再扫描内表匹配。
   适用场景：
   连接表没有合适索引；
   内存可以容纳外表的一部分块；
   比普通嵌套循环减少 I/O。
3. 排序归并连接 Sort-Merge Join
   先把两个输入按连接键排序，然后像归并排序一样线性匹配。
   适用场景：
   两个表都比较大；
   连接条件是等值或范围条件；
   输入本来已经有序；
   查询结果还需要按连接键排序。
   优点是适合大表连接，且输出有序。
   缺点是如果输入无序，需要额外排序成本。
4. 哈希连接 Hash Join
   通常分为两步：
   先对较小的表建立哈希表；
   再扫描较大的表，用连接键去哈希表中查找匹配项。
   适用场景：
   大表等值连接；
   没有合适索引；
   连接结果较大；
   OLAP 查询常见。
   优点是大规模等值连接效率高。
   缺点是只适合等值连接；如果哈希表放不进内存，可能需要分区和写临时文件。
5. 索引连接 Index Join
   利用内表的索引进行连接。它本质上常常是 Nested Loop Join 的一种优化形式。
   适用场景：
   外表过滤后结果很小；
   内表连接键有高效索引；
   每次查找内表代价低。